



THE UNIVERSITY OF HONG KONG

Reinforcing Retrieval-Augmented Generation with Reasoning-Guided Queries and Verifiable Output

Final Year Project Plan

Supervisor: Chao Huang

Lian Tuzhi

3036065159

Chen Borun

3036052190

Table of Contents

Project Background.....	3
Project Objective.....	4
Project Methodology.....	5
1. Reasoning-aware Query Reformulation	6
2. Hybrid retrieval, Re-ranking, and Evidence Packaging.....	6
3. Evidence-bounded Generation with Transparent Citations	6
4. Post-generation Verification and V-Score	7
5. Evaluation and Analysis	7
Project Schedule and Milestones	8

Project Background

Large Language Models (LLMs) have shown strong performance across knowledge-intensive tasks, yet they continue to suffer from factual inconsistencies and hallucinations. Retrieval-Augmented Generation (RAG) was introduced to mitigate these problems by using model outputs in external knowledge sources. However, existing RAG workflows seem to face two limitations. First, retrieval quality is not always aligned with reasoning needs: models often retrieve passages that are topically relevant but not enough to support multi-step reasoning or fine-grained verification (Wang et al., 2025). Then, generated outputs may cite evidence superficially while still containing unsupported claims, leading to a lack of transparency and trustworthiness (Wei et al., 2025; Zhang et al., 2025).

Recent research highlights that RAG systems can be improved by more tightly coupling reasoning and retrieval (Wang et al., 2025; Li et al., 2025). On the one hand, reasoning-guided query reformulation can help the retriever capture the true intent behind complex queries. Also, post-generation verification mechanisms can evaluate whether answers are genuinely supported by the retrieved evidence. This project explores how to reinforce the reliability of RAG by combining reasoning-aware retrieval and verifiable output scoring, aiming to construct a pipeline that not only produces correct answers but also allows users to easily audit their factual basis.

Project Objective

The main objective of this project is to design and implement a RAG framework that enhances both retrieval adequacy and output verifiability. Specifically, the project seeks to

1. Integrate chain-based reasoning techniques to guide query decomposition and reformulation, ensuring that retrieval captures the most relevant and logically necessary evidence.
2. Employ hybrid dense-sparse retrieval with re-ranking and progressive selection to improve coverage and reduce redundancy in the evidence set.
3. Develop an evidence-bounded generation mechanism where the model explicitly grounds its outputs in the retrieved context, abstaining when evidence is insufficient.
4. Introduce a verifiability scoring module that decomposes the generated answer into atomic claims, aligns each claim with evidence, and produces an interpretable reliability score.

By achieving these objectives, the project aims to demonstrate measurable improvements in retrieval precision, answer correctness, and citation quality compared with a traditional RAG baseline. The product will not only be a functional prototype system but also a reproducible evaluation suite and a minimal web demo, allowing others to interactively explore the process from query reformulation to evidence selection, answer generation, and verifiability assessment.

Project Methodology

Our approach follows a modular pipeline structure. This design ensures that improvements introduced at each stage can be evaluated in isolation before being integrated into the whole system. The starting point is a baseline framework that is fully reproducible and instrumented with logging, token-level accounting, and standard relevance and factuality metrics. In addition to this baseline, we incorporate three dedicated modules: reasoning-aware query reformulation, hybrid retrieval with re-ranking and evidence packaging, and evidence-bounded generation with post-generation verification.

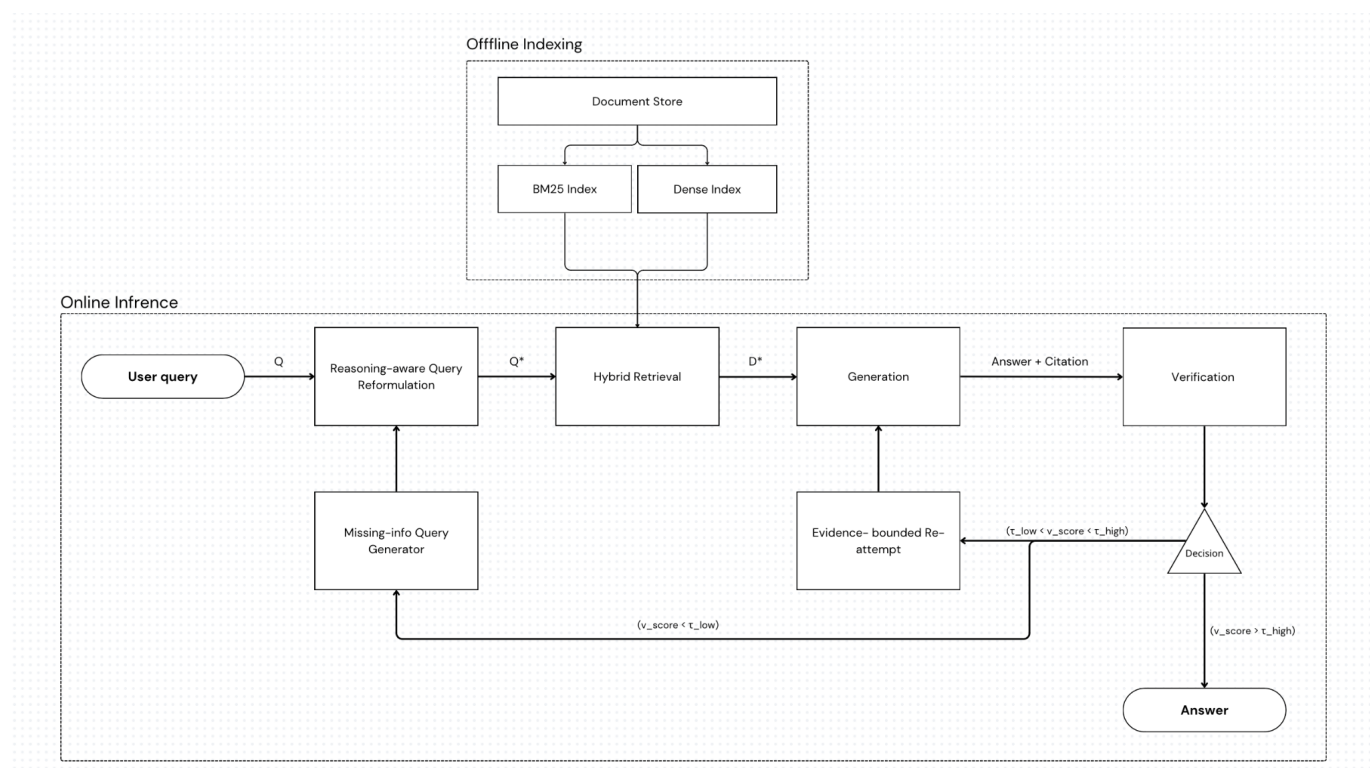


Fig. 1 System architecture

1. Reasoning-aware Query Reformulation

For each user question, we first sketch a short reasoning note whose only job is to steer the search. In two to four sentences, we break down what the question is really asking, name the key entities, pin down any time ranges, and make the relations clear. We also resolve vague terms—for example, when a multi-step answer is needed, we suggest small sub-queries. We then turn those sub-queries into one to three concise search queries. To keep things focused, we only expand with common aliases and clear constraints such as dates or locations, and we drop speculative rewrites that the original question does not justify. We cache successful rewrites by entity and record which rewrite led to the best evidence, so we avoid repeating work and can study failures later.

2. Hybrid retrieval, Re-ranking, and Evidence Packaging

Reformulated queries hit both sparse (BM25) and dense (Embedding) indexes. We take the Top-N from each, merge, and de-duplicate by URL, passage hash, and high Jaccard overlap. Then, a lightweight re-ranker will score each passage against the original question together with the reasoning notes, favoring snippets that jointly maximize relevance and complementarity by covering different aspects rather than repeating the duplicate content. To keep the context focused, a budgeted packer trims passages to the smallest self-contained spans (60 to 120 words) and groups them by aspect, selecting them in a round-robin manner so that no single source dominates. The packer enforces a strict token budget and an explicit diversity quota across sources. If the evidence remains weak after re-ranking, and few high-scoring passages are found or intense contradictions persist, the pipeline issues one additional retrieval round guided by the missing aspects extracted from the reasoning notes.

3. Evidence-bounded Generation with Transparent Citations

The generator receives only the question and the curated evidence pack. It must write the answer from these passages and add inline citations at the sentence or clause level. If part of the answer cannot be supported, it says so briefly rather than guessing. We decode with a low

temperature and give light coverage hints, so the model addresses the sub-questions in order. Each citation links to a list beneath the answer (i.e. title, link or document ID, short snippet), so readers can check the grounding immediately.

4. Post-generation Verification and V-Score

After the answer is generated, a verifier breaks it into small, checkable claims and tests each one against the evidence. We use two complementary checks. One looks for entailment or contradiction to catch unsupported leaps and outright conflicts. The other checks lexical and semantic overlap to confirm that key entities, numbers, and relations really appear in the cited text. Each claim gets a label (supported, weakly supported, not supported, or contradicted), and we combine these into a single verifiability score from 0 to 100. The score balances two factors: the number of claims backed and the frequency of irrelevant text citations. If the score falls below a threshold (, we attempt one automatic repair: find the missing or disputed claims, retrieve targeted passages, and regenerate only those lines. If that still does not clear the bar, we keep the low score and explain why, so users can decide whether to trust the result or revisit it.

5. Evaluation and Analysis

Each module is evaluated both in isolation and within the whole pipeline. Retrieval quality is assessed before generation, while end-to-end evaluation measures factuality of the final answer together with the verifiability score. Improvements over the baseline are reported with confidence intervals, and latency and token-cost footprints are provided for each module and each iteration. Ablation studies remove one component at a time, including the reformulation step, dense or sparse retrieval, re-ranking, diversity controls in packaging, evidence constraints in generation, and the verification layer. Stress tests introduce distractor passages, ambiguous entities, and conflicting sources to verify that the system abstains rather than hallucinating. All prompts, thresholds, and heuristics are recorded in configuration files for reproducibility. The deliverable includes a small web demo that exposes the whole trace, from queries to passages to the answer, citations, and the verifiability score.

Project Schedule and Milestones

Month	Milestone & Deliverables	Key Tasks	Est. Learning Hours
September	Implement traditional RAG baseline	Literature Review and Baseline Setup	16
October	Reasoning-Guided Query Reformulation	Design and implement chain-based query decomposition and missing-information query generation.	24
November	Hybrid Retrieval & Progressive Selection Module	Integrate dense retrievers, re-rankers, and selection strategies	40
December	Evidence-Bounded Generation	Implement a controlled generation module with citation support and abstention mechanisms.	35
January	Verifiability Module	Develop the claim extraction and evidence matching components.	26
February	End-to-End Integration	Connect all modules into a single pipeline	20
March	Evaluation	Run experiments on at public QA datasets	40
April	Web & Video & Poster Creation	Design website, video and poster for exhibition	28
May	Exhibition		24

References:

Wang, Y., Zhao, S., Fan, M., Zhang, Y., Zhang, Z., Wang, Z., ... & Liu, T. (2025). RAG+: Enhancing Retrieval-Augmented Generation with Application-Aware Reasoning. arXiv preprint arXiv:2506.11555. <https://doi.org/10.48550/arXiv.2506.11555>

Wei, J., Zhou, H., Zhang, X., Zhang, D., Qiu, Z., Wei, W., ... & Ouyang, W. (2025). AlignRAG: An Adaptable Framework for Resolving Misalignments in Retrieval-Aware Reasoning of RAG. arXiv preprint arXiv:2504.14858. <https://doi.org/10.48550/arXiv.2504.14858>

Li, D., Wang, Z., & Zhou, H. (2025). Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs. arXiv preprint arXiv:2507.09477.